

Restaurant Management & POS — MVP Technical Specification

Version 1.1 · 11 September 2026 · Revised after gap review — supersedes v1.0

Contents

Revision History	2
1. Project Scope	2
2. Recommended Technology Stack	3
3. Database	4
4. Caching Architecture	5
5. Menu Synchronization	5
6. Offline POS	6
7. POS Authentication	8
8. Roles and Permissions	9
9. Authentication Sessions	10
10. POS Sessions	10
11. POS Hardware — Printers	11
12. Biometric Devices	12
13. Order Lifecycle & Processing	12
14. Voids, Refunds, Discounts & Comps	14
15. Menu & Inventory	15
16. COGS	16
17. Money, Tax & Business Day Rules	17
18. Sales & Tax	18
19. Purchases	18
20. Expenses	19
21. Finance vs Accounting	19
22. Double-Entry Accounting	20
23. Chart of Accounts	21
24. Journal Structure & Posting Rules	21
25. Profit Calculation	23
26. Reporting	24
27. Heavy Reports	25
28. Redis (Deferred)	25
29. Operations	26
30. Overall Architecture	26
31. MVP vs Future	28
32. Final Technology Decision	29
33. Open Decisions	31
The Core Philosophy	31

Revision History

Version	Date	Summary
1.0	—	Initial technical agreement
1.1	11 Sep 2026	Gap review: order lifecycle, offline rules, accounting fixes, controls, security, money and tax rules, inventory detail, operations; Redis and server push deferred

What changed in v1.1

Area	Change	Sections
Order flow	Order and item states, dine-in and takeaway, tables, kitchen tickets, split and merge bills; the database transaction now runs at payment	13
Offline	Offline sales are recorded facts; device-scoped invoice numbers; offline PIN rule; persistent browser storage	6
Accounting	COGS posted on every sale; expanded chart of accounts; posting rules for every business event; permanent records	22–24
Controls	Voids, refunds, discounts and comps with reason codes and owner approval	8, 14
Security	POS device registration, PIN lockout, auto-lock	7, 9
Money and tax	Integer money storage, tax mode, one rounding rule, business day	17
Inventory	Units of measure, weighted-average cost, modifiers in recipes, stock counts, waste	15, 16
Cash sessions	Pay-ins and pay-outs, business date, end-of-day report, Cash Over/Short	10
Operations	Backups, print queue, cash drawer, monitoring, automated tests	11, 29
Simplified	Redis and server WebSocket deferred; menu sync downloads a full snapshot when the version changes	5, 28, 30
Open decisions	Questions to answer before or early in the build, each with a recommended default	33

Sections marked **NEW** or **REVISED** changed in this version. Unmarked sections are unchanged from v1.0.

1. Project Scope **REVISED**

The first version is intentionally simple:

```
One Restaurant
├── Owner / Admin
├── One Cashier
├── One Waiter
└── One POS Device
```

Service model: **dine-in (with tables) and takeaway**. Delivery comes later. How the waiter enters orders with a single POS device is an open decision (section 33).

The goal is to build a **production-quality foundation without over-engineering the MVP**.

The architecture should be simple now but structured so that we can later add:

- Multiple branches
- Multiple POS terminals
- More employees
- Managers
- Kitchen
- Advanced permissions
- Multi-tenant support
- More advanced accounting
- Offline synchronization at scale

2. Recommended Technology Stack

Application

SvelteKit + TypeScript

SvelteKit will handle both:

- Frontend/UI
- Backend/server-side application logic

Runtime

Node.js

Initial production runtime:

```
SvelteKit
  ↓
Node.js
```

We discussed Bun as an alternative. SvelteKit can work with Bun, but for this project the initial recommendation is **Node.js** because of its mature ecosystem and production compatibility.

Bun can be evaluated later if there is a concrete reason to switch.

3. Database REVISED

PostgreSQL

PostgreSQL will be the **primary source of truth**.

It will contain:

```
Users
Roles
Permissions
POS Devices

Products
Categories
Modifiers
Recipes
Units of Measure
Inventory Items
Stock Movements
Stock Counts

Dining Tables
Orders
Order Items
Payments
Invoices
Discounts
Voids & Refunds

Expenses
Purchases
Cash Pay-ins / Pay-outs

Accounting Accounts
Journal Entries
Journal Entry Lines

POS Sessions
Audit Logs
```

Data integrity rules

- **Money** is stored as integers in the smallest currency unit (e.g. cents), never as floating-point numbers. See section 17.
- **Posted records are permanent.** Paid orders, invoices, payments, stock movements and journal entries are never updated or deleted. A mistake is corrected with a reversing record plus a new, correct one.
- **Inventory is a ledger.** Stock on hand is the sum of stock movements. A cached quantity may exist for speed, but the movements are the truth.
- **Journal entries must balance**, and the database itself enforces it (a constraint checked at commit), not only application code.
- **Sensitive actions are audit-logged:** logins, failed PINs, voids, refunds, discounts, comps, approvals, cash drawer opens and price changes.

ORM

Drizzle ORM

So:

```
SvelteKit
  ↓
Drizzle
  ↓
PostgreSQL
```

4. Caching Architecture REVISED

We agreed that the POS should **not download the entire menu from the database every time the POS screen opens**.

Instead, we will use multiple levels of caching.

Browser cache

HTTP caching can be used for relatively static resources/data.

IndexedDB

For POS data, the more important layer is:

```
Browser
  ↓
IndexedDB
```

We can locally store:

- Products
- Categories
- Menu
- Modifiers
- Prices
- Tax configuration
- Restaurant settings
- Dining tables
- Employee list for PIN login
- Locally created orders
- Pending sync queue

5. Menu Synchronization REVISED

We will use **versioning** so the POS doesn't download the menu when nothing has changed.

For example:

```
Server Menu Version: 182
Local Menu Version: 182
```

No update is required.

If the versions differ:

```
Server: 183
Local: 182
```

the POS downloads the **full menu snapshot** and replaces its local copy:

```
GET /api/menu/version → 183
GET /api/menu         → full menu snapshot
```

A restaurant menu is small (tens of kilobytes), so a full snapshot is fast and avoids the bugs of change-only sync, such as tracking deleted items. HTTP caching (`ETag`) makes the version check nearly free.

Change-only sync (`GET /api/menu/sync?from=182`) moves to **Later**, for when there are many terminals or very large menus.

6. Offline POS REVISED

The POS should be capable of continuing to work when the Internet temporarily disappears.

We agreed to use:

- Service Worker
- IndexedDB
- Local sync queue
- Idempotency keys
- Conflict handling

Architecture:

```
POS UI
  ↓
IndexedDB
  ↓
Pending Sync Queue
  ↓
Internet returns
  ↓
SvelteKit API
  ↓
PostgreSQL Transaction
```

Important principle

The browser is **not the final source of truth**. The server owns the menu, prices, inventory, accounting and reports.

The one exception is a **completed offline cash sale**: it has already happened, so the server must accept it as a fact (below).

How much offline work is needed depends on hosting (section 33). With a cloud server, offline support matters a lot; with a server inside the restaurant, outages are rarer.

Offline sales are recorded facts

Once a cash sale is completed offline and the receipt is printed, the sale has happened. When the POS syncs, the server **records** the sale; it does not treat it as a request it can reject.

- **Price at time of sale wins.** Each order line stores the price and tax rate used. A menu price change made while the POS was offline does not change the sale.
- **Stock may go negative.** A sale is never rejected because of stock levels; negative stock is flagged in reports.
- **Problems are flagged, not dropped.** If a synced sale fails validation, it is stored and flagged for owner review, never discarded.
- **Idempotency keys.** Every operation carries a unique ID generated on the device, so the server ignores duplicates when a sync is retried.

Invoice numbers

The POS assigns invoice numbers from its own local sequence, whether online or offline:

```
POS1-000001  
POS1-000002  
POS1-000003
```

The sequence is gap-free per device, and the server enforces uniqueness on (device, number). A second terminal later simply gets its own prefix (POS2-...).

Employee login while offline

- The employee who is logged in keeps working.
- Switching employees offline uses PIN hashes cached on the registered device (slow, salted hashes, refreshed on each sync).
- Offline logins are recorded locally and synced to the audit log.

Protecting unsynced data

- The POS calls `navigator.storage.persist()` so the browser does not evict its IndexedDB data.
- The number of unsynced operations is always visible on screen.
- Logout and POS session close are blocked while the sync queue is not empty. Closing a session requires a connection, because reconciliation runs on the server.

What stays online-only

- Online card/mobile payments should **not automatically be treated as successful offline** unless the payment provider/terminal explicitly supports offline authorization.

- The management dashboard (menu edits, purchases, expenses, reports) requires a connection.

7. POS Authentication REVISED

Because the MVP has only one POS device, we don't need complex device-management infrastructure.

We separate normal application login from POS login.

Owner/Admin

Uses:

```
Email
Password
```

and enters the management dashboard.

Cashier / Waiter

The POS displays:

```
Select Employee

Cashier
Waiter

PIN
****
```

The employee selects their account and enters their personal PIN.

POS device registration

Only a registered device may show the PIN screen:

```
Owner logs in on the POS device (email + password)
↓
"Register this device as POS1"
↓
Server issues a long-lived device cookie (HttpOnly, Secure)
↓
PIN login accepted only from registered devices
```

The owner can revoke a device from the dashboard. This is minimal device management, not the complex kind excluded from the MVP.

PIN rules

- PINs are 4–6 digits, stored only as slow salted hashes (e.g. Argon2 or bcrypt).
- After 5 wrong attempts, the employee is locked out for 5 minutes and an audit event is logged.

- The POS returns to the employee selection screen after a set idle time (default 2 minutes, configurable), and optionally after each payment.
- The owner also has a POS PIN, used to approve sensitive actions (section 8).

8. Roles and Permissions REVISED

We will not rely only on the frontend to enforce roles.

The basic model is:

```
User
↓
Role
↓
Permissions
```

For example:

Cashier

```
pos.sell
pos.payment
pos.print_receipt
pos.void_unsent_item
pos.cash_payout          (up to a set limit)
```

Waiter

```
pos.create_order
pos.view_menu
pos.modify_order
pos.send_to_kitchen
pos.transfer_table
```

Actions that require owner approval

Refunds are **no longer a plain cashier permission**. These actions require the owner to enter their PIN on the POS:

```
Refund (full or partial)
Void an item already sent to the kitchen
Discount above the set limit
Comp / staff meal
Re-open a paid order
Open the cash drawer without a sale
Cash pay-out above the set limit
```

The action, the employee, the approver and the reason are stored together in the audit log.

Server enforcement

If a waiter tries to call an unauthorized API directly:

```
POST /api/refunds
```

the server returns:

```
403 Forbidden
```

So hiding buttons in the frontend is **not** considered security.

9. Authentication Sessions REVISED

We agreed to use secure cookie-based sessions.

Conceptually:

```
Browser
  ↓
HttpOnly + Secure Cookies
(user session + registered POS device)
  ↓
SvelteKit
  ↓
Authenticated User + Registered Device
  ↓
Role + Permissions
```

We do not need to put authentication tokens in `localStorage`.

Cookies use `SameSite`, and state-changing requests keep SvelteKit's built-in origin check (CSRF protection) enabled.

10. POS Sessions REVISED

Authentication and a POS shift/session are separate concepts.

Example:

```
Cashier logs in
  ↓
Start POS Session
  ↓
Opening Cash = $500
  ↓
Cashier operates POS
  ↓
Close POS Session
  ↓
Count Cash
  ↓
Reconciliation
  ↓
Print End-of-Day Report
  ↓
Logout
```

Example:

Opening Cash	\$500
Cash Sales	+\$1,500
Cash Refunds	-\$100
Cash Pay-outs	-\$50
Cash Pay-ins	\$0

Expected Cash	\$1,850

If the cashier physically counts:

Actual Cash = \$1,840

then:

Difference = -\$10

The system records the shortage and posts it to the **Cash Over/Short** account (section 24).

This gives us a useful audit trail.

Pay-ins and pay-outs

Cash taken from the drawer for small expenses (for example, paying for an ice delivery) is recorded at the POS as a **pay-out** with a reason and an expense category. It becomes an expense entry automatically. Cash added to the drawer (for example, more change) is a **pay-in**.

Business date

Every POS session belongs to one business date. A sale made at 01:30, in a session that opened the previous evening, belongs to the previous business day. Reports use the business date, not the calendar date.

Session close

- Requires a connection and an empty sync queue.
- Prints an end-of-day report: sales by payment method, discounts, voids, refunds, comps, pay-ins and pay-outs, and expected vs actual cash, broken down by employee.

11. POS Hardware — Printers REVISED

We agreed **not to depend on direct browser-to-printer communication** as the primary architecture.

Instead:

```

SvelteKit POS
  ↓
Local WebSocket / HTTP
  ↓
Local Print Agent
  ├── Receipt Printer (ESC/POS)
  ├── Kitchen Printer (ESC/POS)
  └── Cash Drawer (opened via the receipt printer)

```

This is more reliable for restaurant thermal printers. The print agent runs on the POS machine or the local network, so printing keeps working offline.

The print agent:

- **Queues jobs** when a printer is offline or out of paper, and prints them when it recovers.
- **Reprints** any receipt or kitchen ticket on request; reprints are marked **COPY**.
- **Opens the cash drawer** on cash payments. Opening it without a sale requires approval and is logged.
- **Prints kitchen tickets** with order number, table, waiter, items, modifiers and notes. Voiding a sent item prints a **VOID** ticket for the kitchen.

12. Biometric Devices

If biometric hardware is added later, we can use the same concept:

```

Biometric Device
  ↓
Local Device Agent
  ↓
HTTP / WebSocket
  ↓
SvelteKit
  ↓
Application

```

We don't need to put device-specific SDK logic directly into the browser.

13. Order Lifecycle & Processing REVISED

Restaurant orders are not a single step. A dine-in table is opened, items are added in rounds and sent to the kitchen, and the bill is paid later, sometimes split.

Order types

- **Dine-in:** linked to a table.
- **Takeaway:** no table; usually paid immediately.
- Delivery comes later.

Order status

Status	Meaning
OPEN	Order in progress; items added in rounds
BILLED	Bill printed; adding items re-opens the order
PAID	Payment recorded and invoice issued; final
VOIDED	Cancelled before payment; reason required
REFUNDED	Full or partial refund after payment; owner approval

Normal path:

OPEN → BILLED → PAID

Item status

Status	Meaning
NEW	Entered but not yet sent; can be changed or deleted freely
SENT	Kitchen ticket printed; removing it requires a void
VOIDED	Removed after sending; reason and owner approval required. If already prepared, it is recorded as waste

Table operations

- The table screen shows free and occupied tables with their open amounts.
- Transfer an order to another table.
- Merge tables into one bill.
- Split a bill by item or equally.
- Split a payment across methods (for example, part cash, part card).

Payment transaction

Adding items and sending them to the kitchen are small, ordinary saves. The full all-or-nothing database transaction runs **at payment**:

```

Order BILLED (or takeaway ready to pay)
↓
BEGIN TRANSACTION
↓
Record Payment(s)          cash / card / mobile, may be split
↓
Finalize Totals             subtotal, discounts, tax, total
↓
Record Invoice Number        from the device sequence: POS1-000123
↓
Deduct Inventory            recipes × quantity, incl. modifiers
↓
Create Invoice
↓
Create Journal Entries       sale + COGS
↓
Mark Order PAID
↓
COMMIT

```

If something fails:

```

ROLLBACK

```

For an offline cash sale, the POS prints the receipt immediately and the same transaction runs on the server when the sale syncs.

This prevents situations such as:

```

Payment recorded
BUT
Inventory not updated

```

or:

```

Order created
BUT
Accounting transaction missing

```

14. Voids, Refunds, Discounts & Comps NEW

These are where restaurant money most often goes missing, so each has clear rules.

Action	When	Money effect	Inventory effect	Approval
Delete item	Before sending to kitchen	None	None	No
Void item	After sending to kitchen	Removed from bill	Waste, if already prepared	Owner
Void order	Before payment	None	Waste for prepared items	Owner, if items were sent
Refund	After payment	Money returned; revenue and tax reversed	None (food is not returned)	Owner

Action	When	Money effect	Inventory effect	Approval
Discount	Before payment	Reduces revenue (Sales Discounts)	None	Owner, above limit
Comp / staff meal	Before payment	No revenue	Deducted; cost goes to Comps expense	Owner

Rules

- **Reason codes are required:** customer changed mind, wrong item entered, kitchen error, quality complaint, or other (with a note).
- **Discounts** can be a percentage or a fixed amount, on one item or the whole order. They are applied **before tax**, so tax is charged on the discounted amount.
- **Refunds** go back to the original payment method. Cash refunds reduce the expected cash in the drawer.
- **Everything is logged** (who did it, who approved it, and why) and appears on the end-of-day report by employee.
- **Approval limits** (for example, discounts above 10%) are restaurant settings (section 33).

The tax treatment of comps and staff meals depends on local rules (section 33).

15. Menu & Inventory REVISED

Products should not simply be treated as individual inventory quantities.

For prepared food, we use recipes.

Example:

```

Burger
├── Bread      1
├── Meat      150g
├── Cheese     1
└── Tomato    30g

```

When one burger is sold:

```

Bread      -1
Meat      -150g
Cheese     -1
Tomato     -30g

```

This allows us to calculate the actual cost of the food.

Units of measure

Each ingredient has a base unit used in recipes, and purchase units with conversions:

Ingredient	Base unit	Purchased in	Conversion
Meat	g	kg	1 kg = 1,000 g
Bread	pcs	bag	1 bag = 12 pcs

Ingredient	Base unit	Purchased in	Conversion
Cola	can	case	1 case = 24 cans

Modifiers affect recipes

Burger + Extra Cheese → Cheese +1 (price +\$0.50)
 Burger – No Tomato → Tomato -30g

Stock movements

Every change in stock is a movement with a type:

Movement	Direction	Created by
Purchase	+	Purchase entry
Sale consumption	–	Paid order (recipe × quantity)
Waste	–	Void after preparation, or waste entry
Comp / staff meal	–	Approved comp at the POS
Count adjustment	+ or –	Stock count

Stock counts

The owner counts stock periodically. The difference between the counted and system quantity is posted as a count adjustment, to the Waste & Inventory Adjustments account.

Negative stock

Sales are never blocked by stock levels (section 6). Negative stock is flagged for the owner to fix; the cause is usually a missing purchase entry or an inaccurate recipe.

Sub-recipes (for example, a sauce made in batches) come later.

16. COGS REVISED

We will use:

Cost of Goods Sold (COGS)

Example:

Burger Selling Price = \$8

Bread	\$0.50
Meat	\$1.50
Cheese	\$0.30
Vegetables	\$0.20

COGS	\$2.50

Therefore:

Revenue	\$8.00
COGS	-\$2.50

Gross Profit	\$5.50

This is much more useful than simply looking at sales revenue.

Costing method: weighted average

Each ingredient's cost is the weighted average of what is in stock, updated on every purchase:

In stock:	10 kg at \$5.00	= \$50.00
Purchase:	10 kg at \$6.00	= \$60.00

New stock:	20 kg	= \$110.00

Average cost = \$110.00 / 20 kg = \$5.50 per kg

Recipe costs, and therefore the COGS of each sale, use the current average cost. COGS is posted to the ledger on every sale (section 24), so gross profit comes straight from the books.

17. Money, Tax & Business Day Rules NEW

Money

- Amounts are stored as **integers in minor units** (cents) in `bigint` columns: \$8.50 is stored as `850`.
- All money arithmetic goes through one shared module. JavaScript floating-point numbers are never used for money.
- Ingredient quantities use fixed-precision decimals (e.g. `numeric(12,3)`).
- The MVP uses a single currency (section 33).

Tax

- **Tax mode** is a restaurant setting. Which one applies is an open decision (section 33):

	Tax-exclusive (10%)	Tax-inclusive (10%)
Menu price	\$10.00	\$11.00
Tax	\$1.00 added on top	\$1.00 included in price
Customer pays	\$11.00	\$11.00
Revenue	\$10.00	\$10.00

- **One rounding rule**, implemented in one function and used everywhere (POS, server, reports). Default: calculate tax at full precision per line and round once on the invoice total. Confirm against local requirements.
- Discounts are applied before tax.
- Each order line stores the tax rate used, so later rate changes don't alter past sales.

Business day and time

- Timestamps are stored in UTC (`timestamptz`); the restaurant's time zone is a setting.
- Reports group sales by **business date** (the POS session's date), so sales after midnight count toward the correct day.

18. Sales & Tax REVISED

Example:

Burger	\$8
Drink	\$2

Subtotal	\$10
Tax	\$1

Total	\$11

Payment:

Cash = \$11

The accounting impact becomes:

Sale:	
Cash	+\$11.00
Sales Revenue	+\$10.00
Tax Payable	+\$1.00
Cost of the food (burger \$2.50 + drink \$0.50):	
COGS	+\$3.00
Inventory	-\$3.00

The tax is **not revenue/profit** for the restaurant.

19. Purchases REVISED

Example:

100 KG Meat
Cost = \$500

Inventory increases:

Inventory +\$500

If paid immediately:

Cash -\$500

If purchased on credit:

Accounts Payable +\$500

When the supplier is paid later:

Accounts Payable -\$500
Cash (or Bank) -\$500

- Purchases are entered in purchase units (e.g. kg) and converted to base units (g).
- Each purchase updates the ingredient's weighted average cost (section 16).
- The **Accounts Payable account is part of the MVP**. Full supplier management (supplier records, statements, ageing) stays in Later.

20. Expenses REVISED

The MVP should support basic expenses such as:

Rent
Electricity
Internet
Salaries
Maintenance
Other Expenses

Example:

Rent	\$500
Electricity	\$100
Internet	\$50
Salary	\$800

These feed into the financial reporting system.

- Each expense category maps to an expense account (section 23).
- Expenses are paid from the bank, or from the cash drawer as a POS pay-out (section 10).

21. Finance vs Accounting

We agreed to distinguish the two.

Finance

Focuses on operational financial information:

- Sales
- Payments
- Cash
- Purchases
- Expenses
- Inventory cost

- Profit
- Financial reports

Accounting

Provides the underlying financial structure:

- Chart of Accounts
- Journal Entries
- Debit
- Credit
- Double-entry accounting

22. Double-Entry Accounting REVISED

We agreed that even though the MVP is simple, the accounting foundation should use **double-entry accounting**.

Every financial transaction has:

Debit
Credit

Example: a customer pays \$11 cash for a burger and a drink (food cost \$3.00):

Entry 1 – Sale
 Debit: Cash \$11.00
 Credit: Sales Revenue \$10.00
 Credit: Tax Payable \$1.00

Entry 2 – Cost of the food
 Debit: COGS \$3.00
 Credit: Inventory \$3.00

Therefore:

Entry 1: Total Debit \$11.00 = Total Credit \$11.00
 Entry 2: Total Debit \$3.00 = Total Credit \$3.00

They must always balance.

Rules

- Journal entries are **generated automatically** from business events (sales, purchases, expenses, session close). The owner never has to type a debit or credit.
- Posted entries are **never edited or deleted**. A mistake is fixed with a reversing entry plus a correct new entry.
- The database rejects any entry whose debits and credits don't match.

23. Chart of Accounts REVISED

The MVP starts with a small Chart of Accounts, which now includes every account the MVP's own transactions need:

```
Assets
1000  Cash on Hand
1010  Bank
1020  Payment Clearing – Card
1030  Payment Clearing – Mobile Money
1200  Inventory

Liabilities
2000  Accounts Payable
2100  Tax Payable

Equity
3000  Owner's Capital
3100  Owner's Drawings
3900  Retained Earnings

Revenue
4000  Sales Revenue
4100  Sales Discounts          (reduces revenue)
4200  Sales Refunds           (reduces revenue)

Cost of Sales
5000  Cost of Goods Sold
5100  Waste & Inventory Adjustments
5200  Comps & Staff Meals

Operating Expenses
6000  Rent Expense
6100  Salary Expense
6200  Utilities Expense       (electricity, water, internet)
6300  Maintenance Expense
6400  Payment Processing Fees
6800  Cash Over/Short
6900  Other Expenses
```

Clearing accounts hold card and mobile payments until the money actually arrives in the bank. Only the payment methods the restaurant accepts are created (section 33).

We can expand this later.

24. Journal Structure & Posting Rules REVISED

The accounting database should conceptually contain:

```
accounts
journal_entries
journal_entry_lines
```

For example:

Sale #1001

↓

Journal Entry 1 – Sale

└─ Cash	Debit	\$11.00
└─ Sales Revenue	Credit	\$10.00
└─ Tax Payable	Credit	\$1.00

Journal Entry 2 – Cost of the food

└─ COGS	Debit	\$3.00
└─ Inventory	Credit	\$3.00

This means we don't simply modify balances manually.

Instead:

Business Event

↓

Financial Transaction

↓

Journal Entry

↓

Account Balances

This is important for auditability.

Posting rules

Every business event maps to a fixed journal entry:

Business event	Debit	Credit
Cash sale	Cash on Hand	Sales Revenue, Tax Payable
Card / mobile sale	Payment Clearing	Sales Revenue, Tax Payable
Cost of food sold	COGS	Inventory
Sale with discount	Cash (or Clearing), Sales Discounts	Sales Revenue, Tax Payable
Clearing settled to bank	Bank, Payment Processing Fees	Payment Clearing
Cash refund	Sales Refunds, Tax Payable	Cash on Hand
Waste / void after preparation	Waste & Inventory Adjustments	Inventory
Comp / staff meal	Comps & Staff Meals	Inventory
Stock count shortfall	Waste & Inventory Adjustments	Inventory
Stock count surplus	Inventory	Waste & Inventory Adjustments
Purchase paid immediately	Inventory	Cash on Hand or Bank
Purchase on credit	Inventory	Accounts Payable
Supplier paid	Accounts Payable	Cash on Hand or Bank
Expense / POS pay-out	Expense account	Cash on Hand or Bank
Cash shortage at session close	Cash Over/Short	Cash on Hand
Cash overage at session close	Cash on Hand	Cash Over/Short
Tax paid to the authority	Tax Payable	Bank

Business event	Debit	Credit
Owner invests money	Cash on Hand or Bank	Owner's Capital
Owner withdraws money	Owner's Drawings	Cash on Hand or Bank

Example: sale with a discount

Burger \$10.00, 10% discount, 10% tax

Menu price	\$10.00
Discount	-\$1.00
Taxable amount	\$9.00
Tax (10%)	\$0.90
Customer pays	\$9.90

Debit:	Cash	\$9.90
Debit:	Sales Discounts	\$1.00
Credit:	Sales Revenue	\$10.00
Credit:	Tax Payable	\$0.90

Total debit \$10.90 = total credit \$10.90.

25. Profit Calculation REVISED

The basic financial model becomes:

Gross Sales	
-	
Discounts and Refunds	
=	
Net Sales	
Net Sales	
-	
COGS (including waste and comps)	
=	
Gross Profit	
Gross Profit	
-	
Operating Expenses	
=	
Net Profit	

Example:

Gross Sales	\$3,100
Discounts	-\$60
Refunds	-\$40

Net Sales	\$3,000
COGS	-\$1,000

Gross Profit	\$2,000
Rent	-\$100
Electricity	-\$50
Salaries	-\$300
Other Expenses	-\$50

Net Profit	\$1,500

26. Reporting REVISED

The system should eventually provide reports such as:

Sales

Daily Sales
 Weekly Sales
 Monthly Sales
 Sales by Product
 Sales by Category
 Sales by Payment Method
 Sales by Order Type (dine-in / takeaway)
 Sales by Employee

POS

End-of-Day Report
 Cashier Sessions
 Opening Cash
 Expected Cash
 Actual Cash
 Cash Difference
 Pay-ins / Pay-outs
 Voids, Refunds, Discounts and Comps by Employee

Inventory

Current Stock
 Stock Movement
 Purchases
 Consumption
 Waste
 Stock Count Differences
 Negative Stock Alerts
 COGS

Finance

```
Revenue
Discounts and Refunds
Expenses
Gross Profit
Net Profit
Cash
Tax
```

Accounting

```
Chart of Accounts
Journal Entries
Debits
Credits
Account Balances
Trial Balance
```

27. Heavy Reports REVISED

We should not eventually calculate massive reports by loading thousands/millions of raw records into the browser.

For the MVP, well-indexed SQL queries on PostgreSQL are enough. As data grows, and only when reports are measurably slow, we can add:

- Summary tables
- Materialized views where appropriate
- Redis caching where appropriate
- Background jobs for expensive reports

28. Redis (Deferred) REVISED

Redis is **not part of the MVP**.

With one restaurant and one POS device, PostgreSQL comfortably handles sessions, menu data and PIN lockout counters, and SvelteKit can cache the menu and settings in memory. Adding Redis now would mean one more service to run, secure and back up, with no measurable benefit.

Redis is added later when there is a concrete need:

```
Multiple app servers    (shared sessions, rate limits)
Background job queues   (heavy reports, notifications)
Heavy read traffic      (menu/settings for many terminals)
```

Critical financial data always remains in PostgreSQL.

29. Operations NEW

Backups

- Automated daily PostgreSQL backups, plus continuous WAL archiving for point-in-time recovery once the system is live.
- Backups are stored off-site, on a different machine or provider from the database.
- **Restores are tested** before launch and then monthly. A backup that has never been restored is not a backup.
- A backup is always taken before running database migrations.

Monitoring

- Uptime check on the server, alerting the owner if it goes down.
- Error logging for the server and the POS (POS errors are sent when online).
- Disk space alerts on the database server.

Deployment

- Docker + Nginx, HTTPS only.
- Updates are deployed outside service hours; database changes go through Drizzle migrations.

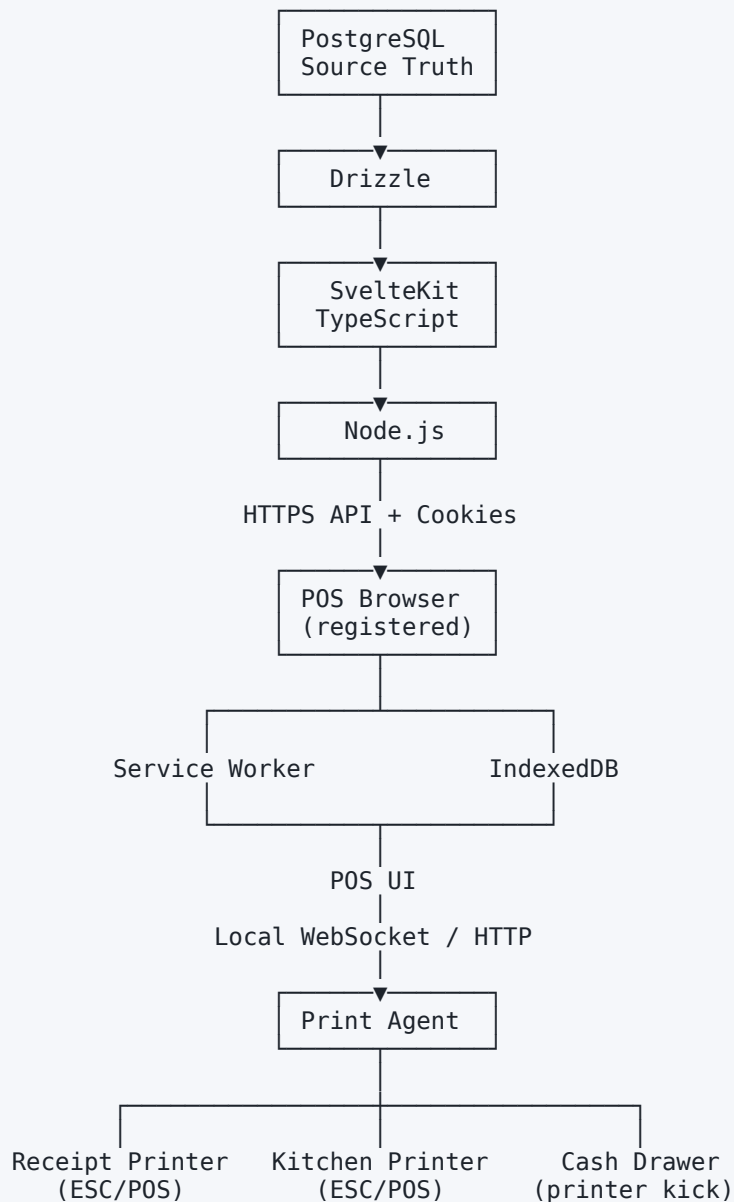
Automated tests

The parts where mistakes cost money get automated tests from day one:

```
Money arithmetic and rounding
Tax calculation (both tax modes)
Journal entries always balance
Posting rules for every business event
Offline sync: retries never create duplicates
Permission checks on every POS API
```

30. Overall Architecture REVISED

Putting everything together:



Changes from v1.0: Redis and the server WebSocket are no longer in the MVP (sections 28 and 32), and the print agent now also drives the kitchen printer and cash drawer. Later additions (extra terminals, Kitchen Display System, biometric device agent) attach to this same structure.

31. MVP vs Future REVISED

MVP

- ✓ One restaurant
- ✓ One branch
- ✓ One registered POS device
- ✓ One cashier
- ✓ One waiter
- ✓ Owner/Admin
- ✓ Email/password for management
- ✓ PIN for POS employees (with lockout)
- ✓ Owner PIN approvals
- ✓ Products
- ✓ Categories
- ✓ Menu and modifiers
- ✓ Dine-in (tables) and takeaway
- ✓ Order and item lifecycle
- ✓ Kitchen tickets (printer)
- ✓ Split and merge bills
- ✓ Payments (split across methods)
- ✓ Invoices (device-scoped numbering)
- ✓ Taxes
- ✓ Discounts, voids, refunds and comps
- ✓ Inventory and stock movements
- ✓ Units of measure
- ✓ Recipes
- ✓ Stock counts and waste
- ✓ COGS (weighted average)
- ✓ Expenses
- ✓ Cash sessions, pay-ins and pay-outs
- ✓ End-of-day report
- ✓ Basic Finance
- ✓ Double-entry Accounting (automatic posting)
- ✓ Accounts Payable account (basic)
- ✓ Reports
- ✓ Browser caching
- ✓ IndexedDB
- ✓ Offline POS (cash sales)
- ✓ Thermal printer integration
- ✓ Automated, tested backups

Later

- Multiple branches
- Multiple POS terminals
- Waiter handheld devices
- Manager role and remote approvals
- Advanced employee management
- Advanced RBAC
- Kitchen Display System
- Delivery orders
- Sub-recipes / batch preparation
- Multiple warehouses
- Supplier management
- Accounts Receivable
- Payroll
- Bank reconciliation
- Advanced accounting
- Change-only menu sync
- Advanced offline synchronization
- Redis (caching, queues)
- Server push (WebSocket)
- Biometric integration
- Advanced analytics
- Multi-tenant SaaS

32. Final Technology Decision REVISED

So, our current agreement is:

Frontend:
SvelteKit + TypeScript

Backend:
SvelteKit server

Runtime:
Node.js

Database:
PostgreSQL

ORM:
Drizzle

Server Cache:
None in MVP (PostgreSQL + in-memory); Redis later

Browser Cache:
HTTP Cache

POS Local Storage:
IndexedDB (persistent storage)

Offline:
Service Worker + Sync Queue + Idempotency Keys

Menu Sync:
Version check + full snapshot

Real-time:
Local WebSocket/HTTP to the Print Agent; server push later

Printing:
Local Print Agent + ESC/POS
(receipt printer, kitchen printer, cash drawer)

Authentication:
Secure HttpOnly Cookies

POS Access:
Registered device + Employee PIN

Management Login:
Email + Password

Authorization:
RBAC + Permissions + Owner PIN approvals

Money:
Integer minor units (cents)

Invoice Numbering:
Device-scoped sequence (POS1-000001)

Accounting:
Double-entry, generated automatically

Inventory:
Recipe/BOM-based consumption, weighted-average cost

Backups:
Automated, off-site, tested restores

Deployment:
Docker + Nginx

33. Open Decisions NEW

These need an answer before or early in the build. Each has a recommended default so work isn't blocked.

#	Decision	Recommended default	Affects
1	How does the waiter enter orders with one POS device?	The waiter uses the shared POS at the counter; a tablet becomes a second registered terminal later	Order flow, device count
2	Cloud hosting or a server inside the restaurant?	Cloud server (Docker + Nginx) with the offline POS, so the owner can reach the dashboard from anywhere	Offline scope, backups, remote access
3	Local tax rules: tax-inclusive or exclusive prices, rounding, legal receipt/invoice requirements, tax on staff meals	Tax mode as a setting; round on the invoice total; confirm with a local accountant	Tax calculation, receipts, invoice numbering
4	Payment methods and currencies at launch	Cash plus one card or mobile-money method; one currency	Clearing accounts, offline rules, reconciliation
5	Who approves refunds and voids when the owner is absent?	Owner PIN only; add a Manager role later if needed	Permissions, daily operations
6	Approval limits and lock timing	Discounts above 10% and pay-outs above a set amount need approval; auto-lock after 2 minutes idle	POS settings
7	Inventory costing method	Weighted average	COGS, inventory value

The Core Philosophy

Keep the MVP simple, but don't make the foundation disposable.

We don't need microservices, multi-branch infrastructure, complicated device management, or a full ERP accounting suite on day one.

But the core foundations (**PostgreSQL transactions, proper authentication, permissions, inventory movements, double-entry accounting, auditability, and local POS caching/offline support**) should be designed correctly from the beginning.